

Guia de Estudos — Rust

Do código da tabuada à vaga real de mercado

Parte 1 — Entendendo o código da tabuada

```
use std::io;

fn main() {
    let mut entrada = String::new();
    println!("Digite um numero:");
    io::stdin().read_line(&mut entrada).unwrap();

    let numero: i32 = entrada.trim().parse().unwrap();

    for i in 1..=10 {
        println!("{}", i, numero, numero * i);
    }
}
```

`use std::io;`

Importa o módulo de entrada/saída da biblioteca padrão. É necessário para ler o que o usuário digita no terminal.

`fn main() { ... }`

Toda aplicação Rust começa pela função main. É o ponto de entrada do programa.

`let mut entrada = String::new();`

Cria uma String vazia e mutável. Por padrão, variáveis em Rust são imutáveis; a palavra mut libera a alteração do valor.

`println!("Digite um numero:");`

Macro (note o !) que imprime texto no terminal. Macros em Rust geram código em tempo de compilação.

`io::stdin().read_line(&mut; entrada).unwrap();`

Lê uma linha digitada pelo usuário e escreve dentro da variável 'entrada'. O &mut; entrada passa uma referência mutável (empréstimo) da variável, sem transferir a posse dela. unwrap() extrai o valor, assumindo que a leitura não falhou.

`let numero: i32 = entrada.trim().parse().unwrap();`

trim() remove espaços/quebras de linha; parse() converte o texto em número inteiro (i32); unwrap() novamente assume que a conversão deu certo.

`for i in 1..=10 { ... }`

Laço que percorre de 1 até 10, incluindo o 10 (o = no ..= inclui o limite final).

`println!("{}", i, numero, numero * i);`

Imprime cada linha da tabuada, substituindo os {} pelos valores na ordem informada.

Conceitos Rust que aparecem neste código pequeno

- **Ownership (posse):** a variável 'entrada' é dona da String que criou. Ao passar &mut; entrada, ela apenas empresta o acesso, sem perder a posse.
- **unwrap() e tratamento de erros:** em Rust, operações que podem falhar retornam um tipo Result. unwrap() extrai o valor de sucesso e interrompe o programa (panic) se houver erro — é uma forma simplificada, adequada para scripts pequenos.
- **Tipagem explícita:** 'i32' informa ao compilador que 'numero' deve ser um inteiro de 32 bits.
- **Macros com '!':** println! não é uma função comum, é uma macro que gera código de formatação de texto em tempo de compilação.

Parte 2 — Sobre a linguagem Rust

O que é Rust

Rust é uma linguagem de programação de sistemas, criada originalmente pela Mozilla, focada em performance, segurança de memória e concorrência segura, sem depender de um garbage collector. É compilada (assim como C e C++), mas com um compilador muito mais rigoroso, que impede uma classe inteira de bugs (ponteiros inválidos, vazamentos de memória, condições de corrida) antes mesmo do programa rodar.

Paradigmas de programação e o Rust

Existem 4 paradigmas clássicos: imperativo/procedural, orientado a objetos, funcional e lógico. Nem todos se aplicam a Rust do mesmo jeito — veja o quanto cada um está presente na linguagem.

Paradigma	Aderência	Como aparece em Rust
Imperativo / Procedural	Base da linguagem	Modelo de execução principal: funções, comandos sequenciais, laços (for, while), condicionais (if) e mutação controlada de estado (via mut). O próprio código da tabuada é 100% imperativo.
Orientado a objetos	Parcial (sem herança)	Tem encapsulamento (structs + impl) e polimorfismo (traits, inclusive dyn Trait), mas não tem herança de classes — traits promovem composição no lugar de hierarquias.
Funcional	Forte influência	Funções de primeira classe, closures, iterators encadeáveis (map, filter, fold), pattern matching, imutabilidade por padrão e tratamento de erros no estilo Option/Result.
Lógico	Não se aplica	Rust não tem suporte nativo a programação lógica (unificação, regras declarativas, backtracking como em Prolog). Esse paradigma não faz parte da linguagem.

Resumo: Rust é multi-paradigma, mas não neutro entre os quatro — sua base é imperativa/procedural, com forte influência funcional na forma de manipular dados e erros. Tem características de orientação a objetos (encapsulamento e polimorfismo via traits), porém sem herança clássica. O paradigma lógico simplesmente não está presente na linguagem.

Outras características importantes da linguagem

Ownership e Borrow Checker: O coração da linguagem. Cada valor tem um único dono; referências ('empréstimos') são verificadas em tempo de compilação para nunca permitir acesso inválido ou simultâneo inseguro.

Segurança de memória sem GC: Diferente de Java/Python (que usam garbage collector) e diferente de C/C++ (que exigem gerenciamento manual arriscado), Rust libera memória automaticamente e com segurança, verificada em tempo de compilação.

Tipagem estática e forte: Erros de tipo são pegos na compilação, não em produção. A inferência de tipos reduz a verbosidade.

Zero-cost abstractions: Recursos de alto nível (iterators, generics, closures) compilam para código tão eficiente quanto equivalente escrito manualmente em baixo nível.

Concorrência segura: As mesmas regras de ownership que evitam bugs de memória também evitam data races em código concorrente — o compilador barra o código antes de rodar.

Onde Rust é usado na prática

- Sistemas operacionais e componentes de baixo nível (ex: partes do kernel do Linux já aceitam código em Rust).
- Ferramentas de linha de comando e utilitários de performance crítica.
- Blockchain e sistemas financeiros (exige segurança e previsibilidade).
- Motores de jogos e WebAssembly.
- Backends de alta performance e sistemas distribuídos (como na vaga estudada na Parte 3).

Parte 3 — Entendendo a vaga real de Rust

Resumo da vaga estudada

O PDF anexo (vaga_rust.pdf) traz uma vaga real de **Software Engineer, Rust** na empresa **Genius Sports**, publicada na plataforma Built In, com faixa salarial de **US\$ 145.000 a US\$ 200.000** (salário base anual), para atuação híbrida em Nova York.

Por que a faixa salarial é essa

Rust exige um investimento de aprendizado maior que linguagens como Python ou JavaScript, principalmente por causa do ownership e do borrow checker. Isso reduz o número de profissionais disponíveis no mercado (oferta menor), enquanto a demanda cresce em áreas que exigem alta performance e segurança (sistemas financeiros, infraestrutura, jogos, streaming de dados em tempo real, como no caso da Genius Sports). Essa combinação de oferta menor e demanda especializada eleva a faixa salarial em comparação com linguagens mais populares e com mais profissionais no mercado.

Por que a stack da vaga faz sentido

Rust	Motor principal: baixa latência e segurança de memória para processar dados em tempo real.
TypeScript / Node.js	Camadas de aplicação e integração mais rápidas de desenvolver, complementando o Rust.
PostgreSQL / GraphQL	Armazenamento e exposição de dados de forma estruturada e flexível.
AWS / Docker / Terraform	Infraestrutura em nuvem, containers e automação de provisionamento.
Apache Pulsar / RabbitMQ / ZeroMQ	Mensageria para processar grandes volumes de eventos em tempo real.
WebAssembly / CUDA	Levar código Rust para o navegador (Wasm) e acelerar cálculos em GPU (CUDA).

O que estudar para chegar a uma vaga assim

- Fundamentos de Rust: ownership, borrow checker, structs, enums e pattern matching (como no código da tabuada, só que mais avançado).
- Tratamento de erros idiomático: Result e Option, em vez de depender sempre de unwrap().
- Concorrência: threads, async/await e a crate Tokio.
- Ferramentas do ecossistema: Cargo (gerenciador de pacotes e build), Clippy (linter) e testes automatizados.
- Um domínio de aplicação: sistemas web (Actix, Axum), sistemas distribuídos, ou WebAssembly, dependendo da vaga desejada.